PropOS × Copilot + Claude 项目开发指南

> 项目: PropOS — 智慧物业资产运营管理平台
>
> 文档用途: 指导如何以 GitHub Copilot（含 Claude 模型）为核心 AI 工具高效开展 PropOS 项目
>
> 更新日期: 2026-04-04

目录

一、AI 工具定位与协作策略

- 1.1 工具角色划分
- 1.2 项目特殊性与 AI 协作重点

二、copilot-instructions.md 最优配置

三、项目开展最佳流程

- 总体节奏（对应 PROJECT_PLAN.md 工期）
- 3.1 阶段 0：架构设计（第 1 周）
- 3.2 阶段 1：M1 资产可视化（第 2-5 周）
- 3.3 阶段 2：M2 租务合同（第 6-11 周）
- 3.4 阶段 3：M3 财务 + M4 工单（第 12-16 周）
- 3.5 阶段 4：M5 二房东穿透（第 17-20 周）
- 3.6 阶段 5：集成测试与 UAT（第 21-26 周）

四、各阶段 AI 使用规范

- 4.1 上下文管理（最重要的习惯）
- 4.2 Agent 模式 vs Chat 模式选择
- 4.3 何时不依赖 AI

五、各模块最佳提示词模板

- M1：资产模块
- M2：合同与递增规则
- M3：NOI 计算
- M4：工单状态机
- M5：二房东权限隔离

六、高效工作习惯与避坑指南

- 6.1 每日工作节奏
- 6.2 常见错误与解决方案
- 6.3 代码质量门控
- 6.4 文档同步约定

附录：推荐 VS Code 配置

一、AI 工具定位与协作策略

1.1 工具角色划分

工具	最适合场景	PropOS 对应任务
Copilot Chat（Agent 模式）	多文件修改、系统设计、跨模块重构	架构设计、数据库 Schema 设计、模块骨架生成、文档生成
Copilot 内联补全	单函数/单文件局部补全	业务逻辑实现、WALE 计算公式、状态机代码
Copilot 编辑模式（Edit Mode）	定向修改已有文件	Bug 修复、接口字段调整、权限逻辑更新

> 核心原则：用 Agent 模式做规划与骨架，用内联补全做实现与填充，用编辑模式做精确修改。三种模式形成流水线，不要混用。

1.2 项目特殊性与 AI 协作重点

PropOS 的以下特性决定了 AI 协作的重心：

项目特性	AI 协作策略
三业态差异大（写字楼/商铺/公寓字段不同）	设计 Sealed Class / Discriminated Union，让 AI 生成带完整 discriminator 的数据模型
复杂计算公式（WALE、NOI、KPI 加权）	提供公式 LaTeX 定义，让 AI 生成带单元测试的纯函数实现
主从两级租赁关系（主合同 → 子租赁）	明确说明递归/树形数据结构需求，让 AI 生成 Repository 层时一并处理级联查询
状态机驱动（合同状态、工单状态）	直接提供状态转移图，让 AI 生成带守卫条件（guard condition）的状态机代码
权限矩阵复杂（7 种角色 + 数据行级隔离）	先让 AI 生成完整 RBAC 矩阵文档，再对照生成中间件代码

二、copilot-instructions.md 最优配置

`.github/copilot-instructions.md` 是 Copilot 在本 workspace 内所有对话的全局系统提示。以下是针对 PropOS 的完整推荐配置：

> 将以下内容追加到现有 `copilot-instructions.md` 的末尾（现有 Markdown 工作流配置保留）。

```
---

## PropOS 项目上下文

### 项目概述
PropOS（Property Operating System）是一套自有混合型物业的内部数字化资产运营管理平台。
管理约 40,000 m²、639 套房源，覆盖写字楼/商铺/公寓三业态。

### 技术栈
- **后端**: Dart（框架视具体选型，遵循 Repository + Service 分层架构）
- **移动端**: Flutter（iOS/Android 双端，主力工具）
- **Web 后台**: Flutter Web 或独立前端（PC 优先响应式设计）
- **微信小程序**: 精简版（仅扫码报修 + 状态查看）
- **数据库**: PostgreSQL（推荐）
- **文档**: Markdown → PDF（通过 scripts/md_to_pdf.sh）

### 核心领域模型

三业态枚举:
- `PropertyType`: `office`（写字楼）/ `retail`（商铺）/ `apartment`（公寓）

关键实体层级:
```

Building → Floor → Unit（资产层）

Tenant → Contract → Invoice（租务财务层）

Contract → SubLease（二房东穿透层）

WorkOrder（工单层）

```
核心计算公式:
- NOI = EGI - OpEx; EGI = PGI - VacancyLoss + OtherIncome
- WALE = Σ(剩余租期ᵢ × 年化租金ᵢ) / Σ(年化租金ᵢ)
- KPI总分 = Σ(指标得分ᵢ × 权重ᵢ)

### 代码规范
- Dart: 遵循 Effective Dart，使用 `freezed` 生成不可变数据类
- 命名: `camelCase`（变量/函数）/ `PascalCase`（类型）/ `snake_case`（数据库列名）
- 每个业务模块目录结构: `models/ / repositories/ / services/ / controllers/（或 bloc/）`
- 测试: 核心计算逻辑（WALE、NOI、KPI 打分）必须有单元测试
- 安全: 租客证件号字段必须标注加密存储注释，API 响应默认脱敏

### Phase 1 模块边界
| 模块 | 状态 |
|------|------|
```

```
| M1 资产与空间可视化 | 含 CAD(.dwg→SVG/PNG) 转换 + 楼层热区状态色块 |
| M2 租务与合同管理 | 含状态机、WALE、租金递增规则配置器 |
| M3 财务与 NOI | 含自动账单生成、NOI 实时看板、KPI 仪表盘 |
| M4 工单系统 | 含 Flutter App 移动端 + 精简小程序 |
| M5 二房东穿透管理 | 含主从两级租赁、外部填报 Web 页、审核流 |

Phase 2 功能（租户门户、门禁、电子签章等）**不在当前开发范围内**，生成代码时不要超前实现。

### 架构约束
1. 所有 API 端点必须经过 RBAC 中间件验证角色权限
2. 二房东相关数据查询必须在 Repository 层加行级数据隔离过滤
3. 证件号、手机号字段在数据库层加密存储，API 层默认脱敏（仅显示后4位）
4. 操作审计日志覆盖：合同变更、账单核销、权限变更、二房东数据提交
```

三、项目开展最佳流程

总体节奏（对应 PROJECT_PLAN.md 工期）

```
阶段 0（第 1 周） → 架构决策 + 环境搭建 ← AI 用于设计评审与文档生成
阶段 1（第 2-5 周）→ M1 资产可视化     ← AI 生成数据模型 + CAD 工具脚本
阶段 2（第 6-11 周）→ M2 租务合同      ← AI 生成状态机 + 递增规则引擎
阶段 3（第 12-16 周）→ M3 财务 + M4 工单 ← AI 生成计算公式 + Flutter UI 骨架
阶段 4（第 17-20 周）→ M5 二房东穿透    ← AI 生成权限隔离逻辑 + 审核流
阶段 5（第 21-26 周）→ 集成测试 + UAT       ← AI 生成测试计划 + 验收检查清单
```

3.1 阶段 0：架构设计（第 1 周）

目标：用 AI 快速完成技术架构决策，输出可执行的技术约定文档。

操作流程：

1. 打开 **Agent** 模式，上下文文件加载 `docs/PRD.md`
1. 输入提示词（复制使用）：

```
基于 @PRD.md，为 PropOS 设计完整的系统架构方案，包含：
    1. 后端 Dart 服务的目录结构（分层架构）
    2. PostgreSQL 数据库 Schema（所有 Phase 1 核心表，含三业态扩展字段设计）
    3. Flutter App 的页面路由结构
    4. RBAC 权限矩阵的代码级实现方案
    5. 二房东数据行级隔离的 Repository 层实现方案
输出为 Markdown 技术架构文档，保存到 docs/ARCH.md
```

1. 输出验收清单（让 Copilot 逐项确认）：

- [ ] 数据库表包含 `property_type` 枚举字段
- [ ] Contract 表有 `master_contract_id` 外键支持二房东关系
- [ ] 审计日志表已设计
- [ ] 证件号字段标注加密存储

3.2 阶段 1：M1 资产可视化（第 2-5 周）

核心难点：`.dwg` → SVG/PNG 转换 + Flutter/Web 热区叠加

AI 使用策略：

- 用 Agent 生成 Python/Dart 的 CAD 转换脚本骨架（基于 `ezdxf` 库）
- 用内联补全实现 SVG 热区坐标计算逻辑
- 用 Agent 生成 639 套房源的 Excel 批量导入解析器

关键提示词：

```
实现一个 Dart 命令行工具，将 .dwg 文件批量转换为分层 SVG。
要求：
- 使用 Process.run 调用 ODA File Converter 或 LibreCAD 命令行工具
- 按楼层分割输出（每层一个 SVG 文件）
- 输出文件命名规范：{building_id}_floor_{floor_number}.svg
- 错误时记录日志但不中断批量处理
```

3.3 阶段 2：M2 租务合同（第 6-11 周）

核心难点：租金递增规则引擎（6种递增类型 + 混合分段）

AI 使用策略：

- 先让 AI 设计**递增规则的数据结构**（JSON Schema 或 Dart sealed class）
- 再让 AI 实现**计算引擎**（纯函数，便于单元测试）
- 最后让 AI 生成**WALE 计算**，引用递增引擎的结果

操作顺序（严格按序）：

```
步骤 1 → 生成 RentEscalationRule sealed class（含6种子类型）
步骤 2 → 生成 RentCalculationEngine（接收规则列表 + 计算日期，返回该日应收租金）
步骤 3 → 生成单元测试（覆盖：固定递增/阶梯/CPI/混合分段）
步骤 4 → 生成 WALECalculator（调用 RentCalculationEngine 获取年化租金权重）
步骤 5 → 生成合同状态机（ContractStateMachine，含守卫条件）
```

> 重要：步骤 1→2→3 必须按序，不要一次提示让 AI 把5步全做，那会导致代码内聚性差。

3.4 阶段 3：M3 财务 + M4 工单（第 12-16 周）

财务模块 AI 策略：

- NOI 计算是纯数学公式，直接提供公式 + 字段定义，让 AI 生成带单元测试的 Service
- 账单自动生成是调度任务，让 AI 生成 Dart 的 Cron Job 骨架（每月定时触发）
- KPI 打分引擎与 WALE 引擎同构，复用相同设计模式

工单模块 AI 策略（Flutter App）：

```
生成一个 Flutter 工单报修页面，要求：
- 支持三级联动选择器：楼栋 → 楼层 → 单元（从后端 API 异步加载）
- 照片上传：最多 5 张，使用 image_picker + 压缩处理（> 1MB 自动压缩至 800KB）
- 紧急程度选择：ChipGroup 单选，三个选项
- 工单状态用 BLoC 管理，状态包含：initial / submitting / success / error
- 提交后跳转至工单列表页并高亮新建工单
```

3.5 阶段 4：M5 二房东穿透（第 17-20 周）

核心复杂度：外部 Web 表单 + 行级数据隔离 + 审核流

AI 使用策略：

- 先生成二房东账号的 JWT 中间件（需包含 `masterContractId` claim）
- 再生成 SubLeaseRepository（所有查询自动注入 `WHERE master_contract_id = ?` 过滤）
- 最后生成审核状态机（pending → approved / rejected）

安全专项提示词：

```
审查以下 SubLease 相关的 API 端点代码，确认：
1. 每个端点都从 JWT 中提取 masterContractId，而非从请求体接受
2. Repository 查询绑定了行级隔离条件
3. 二房东账号无法访问合同管理、财务、工单等其他模块 API
4. 操作审计日志记录了操作人、IP、变更前后内容
如有安全漏洞，直接修复并说明原因。
```

3.6 阶段 5：集成测试与 UAT（第 21-26 周）

让 AI 生成验收检查清单（提示词）：

```
基于 @PRD.md 第七节"Phase 1 验收标准"，生成一份完整的 UAT 测试用例表，格式为 Markdown 表格，包含列：
用例编号 | 所属模块 | 测试场景 | 前置条件 | 操作步骤 | 预期结果 | 实际结果（留空） | 通过/失败（留空）
覆盖所有 5 个 Phase 1 模块，包括边界条件（如：639 套批量账单生成 < 30 秒）
```

四、各阶段 AI 使用规范

4.1 上下文管理（最重要的习惯）

规则	说明
每次对话开头引用关键文件	用 `@PRD.md` 、 `@ARCH.md` 等让 Copilot 加载项目上下文
一次对话聚焦一个模块	不要在同一对话里混合讨论 M1 和 M3，会导致上下文污染
生成代码前先确认数据结构	让 AI 先输出数据模型/接口定义，确认无误后再生成实现代码
复杂逻辑分步生成	租金递增引擎、KPI 打分等复杂逻辑，务必分步骤生成，逐步验证

4.2 Agent 模式 vs Chat 模式选择

```
需要修改多个文件 → Agent 模式
需要探索性设计讨论 → Chat 模式（不要让它直接改文件）
需要局部代码补全 → 内联模式（Tab 补全）
需要定向修改某个函数 → Edit 模式（Cmd+I）
```

4.3 何时不依赖 AI

场景	建议
数据库 Schema 最终确认	人工审查，特别是索引设计和外键约束
安全敏感逻辑（JWT、加密）	AI 生成后必须人工 Code Review
业务规则边界（如：免租期计算）	对照 PRD 原文逐字确认，AI 容易理解偏差
性能关键路径（639 条批量处理）	AI 给方案，人工压测验证

五、各模块最佳提示词模板

以下提示词可直接在 Copilot Chat 中使用，根据实际情况替换 `[占位符]` 。

M1：资产模块

```
# 生成房源批量导入解析器
实现一个 Dart Service：ExcelImportService，解析写字楼/商铺/公寓三张 Excel 模板，
模板列定义：[参照 PRD M1 数据初始化方案]
要求：
```

```
- 按业态路由到不同的字段校验规则（PropertyType enum）
- 错误行汇总到 ImportResult.errors，不中断整体导入
- 成功后批量插入数据库，使用事务保证原子性
- 重复单元编号以最新行覆盖旧行（upsert 语义）
```

M2：合同与递增规则

```
# 生成租金递增引擎
使用 Dart 实现 RentEscalationEngine，支持以下 6 种 EscalationRule：
1. FixedPercentageRule(double percentage, EscalationPeriod period)
2. FixedAmountRule(double amountPerSqm, EscalationPeriod period)
3. SteppedRentRule(List<RentStep> steps) // 每个 step 包含 startMonth,
endMonth, rentPerSqm
4. CpiLinkedRule(Map<int, double> yearToCpiMap) // key: 年份, value: CPI%
5. EveryNYearsRule(int n, double percentage)
6. PostFreeRentBaseAdjustmentRule(double baseRentPerSqm)

计算方法签名：
double calculateRent(DateTime targetDate, DateTime leaseStartDate, double
initialRent, double areaInSqm)

输出包含完整单元测试，测试覆盖每种规则类型和混合分段场景。
```

M3：NOI 计算

```
# 生成 NOI 计算 Service
实现 NOICalculationService，方法签名：
NOISummary calculateNOI({
  required DateRange period,        // 计算区间
  String? buildingId,               // null = 全部楼栋
  PropertyType? propertyType,       // null = 全业态
})

NOISummary 包含：
- pgi: double （潜在总收入）
- vacancyLoss: double （空置损失，按市值租金估算）
- egi: double （有效总收入）
- opex: OperatingExpenses （运营支出明细）
- noi: double （净营运收入）
- occupancyRate: double

OperatingExpenses 包含：utility, cleaning, maintenance, insurance, tax 五类。
所有计算必须支持按业态拆分（写字楼/商铺/公寓分别输出）。
```

M4：工单状态机

```
# 生成工单状态机
用 Dart 实现 WorkOrderStateMachine，状态流转如下：
```

```
submitted → (审核通过) → assigned → in_progress → pending_inspection →
completed
submitted → (审核拒绝) → rejected
in_progress → (挂起) → on_hold → in_progress

每个转换方法:
- 校验权限（谁能触发该转换，参照 PRD 角色矩阵）
- 记录审计日志（操作人、时间、状态变更）
- 返回 Either<WorkOrderError, WorkOrder>

同时生成对应的 BLoC（WorkOrderBloc）用于 Flutter 端状态管理。
```

M5：二房东权限隔离

```
# 生成行级数据隔离 Repository
实现 SubLeaseRepository，所有数据库查询必须遵守:
1. 从注入的 SubleaseSecurityContext 获取 masterContractId（不接受参数传入）
2. 所有 SELECT/UPDATE/DELETE 自动追加 WHERE master_contract_id =
$masterContractId
3. INSERT 时自动设置 master_contract_id = $masterContractId
4. 尝试访问其他 masterContractId 的数据时抛出 UnauthorizedException

同时实现 SubleaseAuditLogger，记录二房东的每次操作（含操作前后的 JSON 快照）。
```

六、高效工作习惯与避坑指南

6.1 每日工作节奏

```
上午（编码时间）:
  → 用内联补全完成昨天设计好的功能实现
  → Copilot Tab 补全细节，不要打断思路去开 Chat

下午（设计时间）:
  → 用 Agent/Chat 模式规划明天的任务
  → 让 AI 评审今天的代码，输出 Code Review 意见

遇到 Bug:
  → 先用 Edit 模式（Cmd+I）让 Copilot 就地修复
  → 修复失败再开 Agent 模式加载更多上下文
```

6.2 常见错误与解决方案

错误模式	症状	解决方案
上下文丢失	AI 生成的代码与项目结构不匹配	每次对话开头必须 `@` 引用关键文件
过度生成	AI 实现了 Phase 2 的功能	在 copilot-instructions.md 明确标注 Phase 2 范围
业务规则错误	递增规则计算逻辑与 PRD 不符	提供公式 + 具体数字示例，而非文字描述
权限漏洞	生成的 API 端点缺少权限校验	使用 M5 安全专项提示词做专项审查
三业态混淆	写字楼字段出现在公寓数据模型中	让 AI 先生成 PropertyType sealed class，强制类型区分

6.3 代码质量门控

每个模块完成后，运行以下 Copilot 审查提示词：

```
对 [模块名] 的代码做安全与质量审查，重点检查：
1. 所有 API 端点是否有 RBAC 权限校验
2. 证件号/手机号是否按约定加密存储和脱敏输出
3. 数据库查询是否有 SQL 注入风险（使用参数化查询）
4. 二房东相关查询是否有行级隔离
5. WALE/NOI/KPI 计算是否有对应单元测试
列出发现的每个问题，并直接给出修复代码。
```

6.4 文档同步约定

每完成一个模块的核心逻辑，立即让 AI 补充架构文档：

```
基于当前实现的 [模块名] 代码，更新 docs/ARCH.md 中对应章节：
- 更新数据库表结构（如有变更）
- 更新 API 端点列表
- 补充关键设计决策说明（为什么这样设计）
然后执行：bash scripts/md_to_pdf.sh docs/ARCH.md
```

附录：推荐 VS Code 配置

在 `.vscode/settings.json` 中添加以下配置以优化 Copilot 体验：

```
{
  "github.copilot.chat.codeGeneration.useInstructionFiles": true,
  "github.copilot.nextEditSuggestions.enabled": true,
  "github.copilot.chat.experimental.temporalContext.enabled": true,
```

```
  "chat.agent.enabled": true,
  "github.copilot.enable": {
    "*": true,
    "dart": true
  }
}
```

本文档随项目进展持续更新。每次更新后执行 `*bash scripts/md_to_pdf.sh docs/COPILOT_GUIDE.md*` 同步 *PDF* 版本。